뜨개더

QA PORTFOLIO

01 뜨개더를 만든 이유

유하은 | DB QA | 개인 프로젝트

뜨개더는 도안을 보면서 단수(완료한 줄 수)를 세고, 메모와 재료를 관리하는 iOS 앱입니다. 뜨개질을 하며 도안과 용어 검색, 단수 기록을 따로 관리해야 했던 불편함에서 시작했습니다. 도안을 보다가 모르는 기술을 찾아보고, 돌아와 작업을 이어갈 수 있도록 만들었습니다.

DB QA에서는 기능과 버그 수정 결과를 주로 테스트했습니다. 앱에서는 화면을 조작한 뒤 API와 저장 데이터까지 어떻게 바뀌는지 확인해 보고 싶었습니다.

테스트하면서 발견한 문제

문제	확인한 내용
도안을 바꾸면 그려 둔 선이 사라짐	교체 취소와 승인 후 도안·선의 상태
목록은 보이는데 새 값이 반영되지 않음	화면의 22, DB의 99, 목록 API의 500 응답
두 기기가 같은 메모를 수정하다 충돌	서버에 남은 값, 미저장 메모, 직접 재저장

뒤에서 이 세 가지 문제를 어떻게 재현하고 확인했는지 설명합니다. 당시 화면과 HTTP·SQL 기록도 함께 연결했습니다.

앱 구성

SwiftUI 앱에서 로컬 저장을 지원합니다. 서버 모드에서는 NestJS API와 Prisma·PostgreSQL을 사용합니다. 초기에는 수동으로 화면을 조작하고 Postman·Charles·SQL로 확인했습니다. 이후 Appium·pytest·Swift 테스트와 GitHub Actions로 반복 실행할 항목을 추가했습니다.

02 테스트 방법

저장한 기록이 사라지는 상황부터 봤습니다

단수와 메모는 작업을 이어가는 데 필요한 기록입니다. 저장 실패, 앱 종료, 두 기기의 동시 수정처럼 이 기록이 사라질 수 있는 조건을 먼저 확인했습니다. 로그아웃하고 다른 계정으로 들어갔을 때 이전 계정의 기록이 보이는지도 살폈습니다.

기능별 기대 동작을 정한 뒤 앱에서 재현했습니다. 화면과 실제 저장값이 다르면 요청·응답과 DB를 대조했습니다. 예를 들어 목록 조회 문제는 DB의 단수를 바꾸고, 새로고침 전후의 화면과 HTTP 응답을 비교해 확인했습니다.

방법	사용한 곳
앱 조작·Appium	도안 교체 취소, 화면 복귀, 충돌 안내와 입력 유지
Postman·Charles·pytest	요청·응답, 통신 실패·지연·재시도
SQL·실제 PostgreSQL 테스트	저장값, 연관 기록의 삭제·복원, 동시 요청과 롤백
Swift 테스트	초안 복구, 계정·프로젝트별 저장 분리

초기 수동 검증은 22건을 설계해 21건을 실행했고, 정정된 집계는 11 PASS·10 FAIL입니다. 원래 실패한 케이스와 수정 후 통과한 결과는 실행 기록에서 따로 볼 수 있습니다.

제가 맡은 일과 AI 활용

앱 기획, 테스트 우선순위와 기대 결과를 정했습니다. 충돌이 나면 입력을 남기고 사용자가 다시 저장하게 하는 기준도 정했습니다. 초기 수동 검증에는 Postman·Charles·SQL을 사용했습니다.

앱·서버 구현과 후속 자동화의 코드 작성·실행·증거 수집에는 AI를 활용했습니다. 뒤에 나오는 후속 자동화 결과를 초기 수동 수행 실적으로 세지 않았습니다.

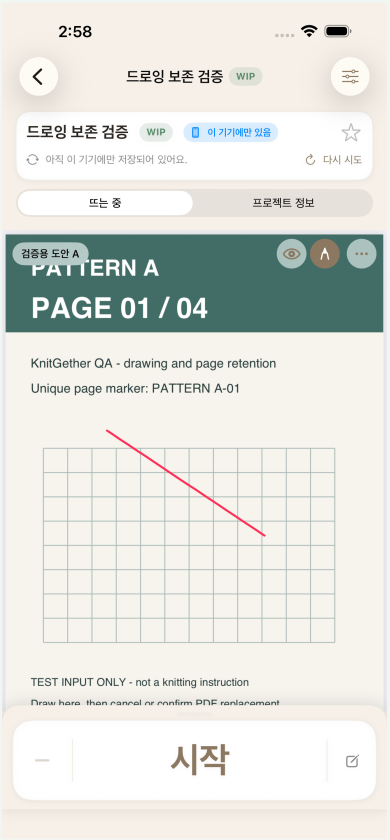
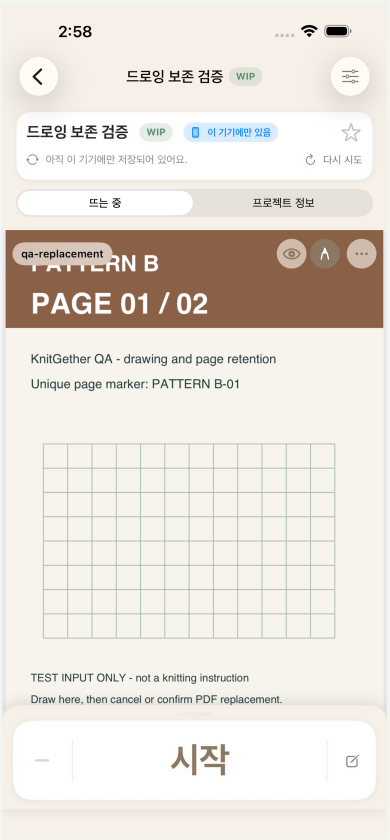
수동 실행 기록 / API·네트워크 실행 기록

03 도안을 바꾸면 선이 사라지는 문제

DEF-10 | 교체 취소와 승인

기존 도안에 선을 그려 둔 상태에서 다른 도안으로 교체했습니다. 새 도안은 열렸지만, 그려 둔 선은 안내 없이 사라졌습니다.

도안 교체 전에 선이 삭제된다는 안내가 필요하다고 정했습니다. 취소하면 기존 도안과 선을 그대로 두고, 승인했을 때만 교체하는 두 경로를 확인했습니다.

취소 후: 도안 A와 선 유지	승인 후: 도안 B로 교체, 선 제거
	

촬영: 2026-10-04 후속 자동화. 로컬 모드의 PDF 직접 교체입니다. 설치 앱의 소스 커밋은 확인하지 못했습니다. 서버 동기화는 이 실행에서 테스트하지 않았습니다.

Appium은 '그리기 삭제' 메뉴가 있는지로 선의 존재를 간접 확인했습니다. 실제로 선이 남았는지 사라졌는지는 PNG 화면도 함께 확인했습니다.

촬영·검사 코드 / 최초 재현 / 화면·실행 결과

04 이전 목록만 남아 있던 조회 오류

DEF-08 | 화면의 22와 DB의 99

프로젝트 하나의 단수 정보만 SQL로 지우고 목록을 새로고침했습니다. 목록 API는 HTTP 500으로 실패했지만, 앱에는 이전 목록이 남아 있었습니다.

정상 프로젝트의 단수를 DB에서 22에서 99로 바꾼 뒤 다시 조회해도 화면은 22였습니다. DB에는 99가 있었고 API는 실패했습니다. 코드를 읽을 때 예상했던 클라이언트 해석 오류에 앞서, 서버가 응답을 만드는 과정에서 이미 실패하고 있었습니다.

정상 항목을 조회한 뒤 오류 항목도 확인했습니다

목록은 단수 정보가 없는 프로젝트를 제외하고 정상 항목을 반환하도록 수정됐습니다. 10월 29일에는 단수 행 삭제와 삭제 상태 변경, 두 조건에서 정상 단수 99가 반환되는지 재검증했습니다. 이 회차에는 앱 화면을 새로 실행하지 않았습니다.

10월 31일에는 오류 항목을 개별 조회하는 경우와 기기에 남은 캐시까지 확인했습니다.

확인한 곳	수정 전	수정·재검증 결과
목록 API	전체 HTTP 500	200, 정상 항목과 제외 ID 반환
오류 항목 개별 조회	HTTP 500	409, 오류 코드로 원인 구분
오류 항목의 기기 캐시	삭제된 항목으로 보고 지울 수 있음	캐시 유지, 최신 조회 불가 안내

개별 조회는 단수 행 삭제·삭제 표시·소유자 불일치에서 409를 반환했습니다. 다른 계정에는 404를 유지했습니다. 조회 전후 단수·메모·작업 세션이 바뀌지 않았고, 시험 전에 확보한 사본으로 복원한 뒤 원래 단수 37과 200 응답을 확인했습니다.

서버에서 제외한 ID는 응답 헤더로 전달합니다. 앱 내부 테스트에서는 해당 캐시와 미저장 메모가 남고, 실제 삭제 항목은 정리되며 정상 단수 99는 갱신되는지 확인했습니다. 디스크에서 다시 읽은 뒤에도 같은 상태인지 확인했습니다.

API·SQL 테스트 / 캐시 테스트 / 재검증 결과

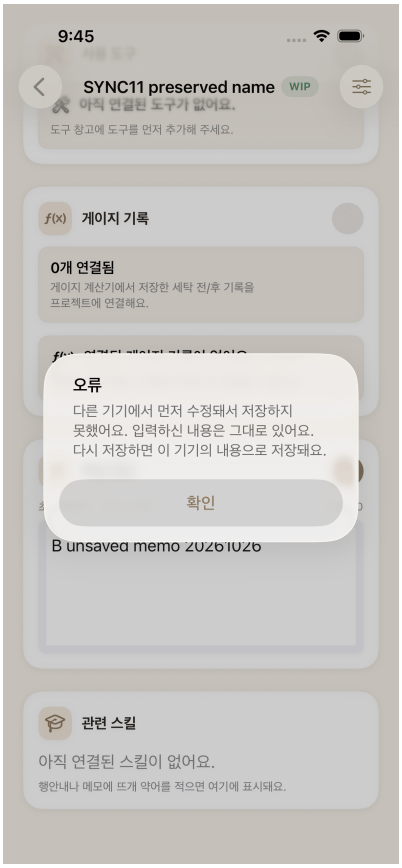
05 두 기기에서 같은 메모를 수정할 때

SYNC-11 | 먼저 저장한 값과 작성 중인 메모

같은 계정의 두 기기 A·B에서 같은 프로젝트를 열었습니다. B에서 메모를 입력해 두고 A에서 먼저 저장한 뒤, B의 내용을 저장하는 순서로 확인했습니다.

서버에는 A의 값을 남기고 B에는 충돌 안내와 미저장 메모를 유지하도록 정했습니다. B가 다시 조회한 뒤 직접 저장하게 하고, 자동 재저장은 하지 않습니다. 초안은 계정·프로젝트별로 기기에 저장하며, 저장 성공이나 명시적 삭제 때 지웁니다.

순서	확인한 결과
B 입력 → A 저장 → B 저장	HTTP 409, DB의 A 메모 유지
B의 충돌 화면	안내와 B 입력 유지
B에서 직접 재저장	DB에 B 메모 저장, 시험한 다른 필드 유지
앱 종료·재실행	초안 복구, 버리기 후 미복구



촬영: 2026-10-26 두 시뮬레이터 후속 자동화. 기본 12개는 UI·API 혼합, 보완 7개는 양쪽 변경을 화면에서 수행했습니다. 자동 재저장 없음은 300초 관찰입니다. 메모·단수 외 모든 입력을 확인한 결과는 아닙니다.

두 기기 화면 테스트 / 실행 결과·HTTP·DB

06 메모 외의 변경도 충돌을 막는지 확인

단수·작업 세션·도안만 바뀐 경우

메모끼리의 충돌이 막혀도 다른 변경이 프로젝트 수정 시각에 반영되지 않으면 오래된 전체 저장이 성공할 수 있었습니다. 이때 요청에 빠진 작업 기록이 지워지거나, 이미 삭제한 기록이 되살아날 수 있습니다.

후속 자동화에서 행 안내 생성·수정·삭제와 PDF·드로잉 변경 뒤 오래된 전체 저장이 200으로 성공하는 6개 실패를 재현했습니다. 다른 프로젝트의 자식 ID를 재사용하는 3개 실패도 확인했습니다.

조건	수정 후 확인한 결과
연관 기록을 바꾼 뒤 오래된 전체 저장	부모 버전과 함께 갱신, 오래된 요청은 409
같은 버전으로 실제 동시 요청	한 요청만 성공, 거절된 요청의 기록은 DB에 없음
처리 중간에 실패	DB와 기존 파일 내용 유지
다른 계정·프로젝트의 자식 ID 재사용	해당 기록 수정 차단

10월 26일 수정 후 실제 HTTP·PostgreSQL 테스트 40개가 통과했습니다. 10월 29일에도 새 전용 DB에서 같은 40개를 재실행해 통과했습니다. 읽기·쓰기는 실제 SQL을 사용하고, 요청 순서와 실패 시점은 테스트 훅으로 제어했습니다. 장시간 부하 테스트는 하지 않았습니다.

도안을 보다가 다른 화면에 다녀온 경우

사전에서 돌아오면 보던 페이지가 유지됐지만, 홈 탭을 거치면 1페이지로 돌아갔습니다. 두 경로 모두 도안을 보던 작업을 이어가는 상황으로 보고, 홈 복귀 때도 페이지를 유지하도록 기준을 정했습니다.

3페이지로 이동한 뒤 본문의 고유 표시를 기준으로 홈·사전 복귀를 확인했습니다. 확대율, 페이지 안의 스크롤 위치, 앱 재실행 후 복원은 이 실행에서 확인하지 않았습니다.

동시성 테스트 / 수정 전후 결과 / 페이지 복귀 코드 / 페이지 복귀 결과

07 실행 결과와 확인하지 못한 부분

2026-10-31 로컬·원격 CI 결과

테스트	결과	확인한 내용
서버 회귀	180 PASS	HTTP 응답, 파일·저장 경로
실제 HTTP·PostgreSQL	40 PASS, 0 SKIP	동시 요청, 롤백, 연관 기록·파일 유지
API·네트워크·DB	79 PASS, 0 xfail	목록 2조건·개별 조회 3조건 포함
Swift	365 PASS / 41 suites	캐시·초안 유지, 오류 구분
시뮬레이터 UI smoke	3 PASS	가입·저장, 오프라인 동기화, 캐시 조회

회귀 실행의 제품 소스는 5bcbaae입니다. 실기기 대화상자 처리를 바꾼 f0663a6은 앱 소스가 같습니다. Windows·Linux의 파일 경로 테스트는 각각 33개 통과했으며 서버 180개에 포함됩니다. 서로 겹치는 테스트 수를 전체 E2E 건수로 합산하지 않았습니다.

앞선 CI에서는 온보딩의 계정 화면 진입이 실패해 2 PASS·1 FAIL이었습니다. 시뮬레이터 부팅 준비와 화면 도착 확인을 보완한 뒤 로컬·원격 UI smoke가 통과했습니다. 최초 실패의 내부 원인은 확정하지 못했습니다.

iPhone에서 확인한 것

iPhone 16 Pro·iOS 27.0.1에서 비로그인 상태로 단수 1을 기록했습니다. 백그라운드에서 돌아오고 앱을 재실행해도 값이 남았습니다. 카메라 권한을 거부한 경우에는 설정에서 허용하라는 안내를 추가한 뒤, 안내를 닫고 작업 화면에 돌아왔을 때도 단수 1이 유지되는지 확인했습니다. 두 흐름은 각각 최종 재실행에서 통과했습니다.

가입·서버 동기화는 앱의 로컬 네트워크 차단으로 끝내지 못했습니다. 재설치·재시동 뒤에도 반복됐고 원인은 아직 확인하지 못했습니다.

도안 교체·페이지 화면은 10월 4일, 두 기기 충돌 화면은 10월 26일 실행입니다. 전 기종 테스트와 장시간 부하는 실행하지 않았으며, 속도 향상률·전체 커버리지·운영 성과도 측정하지 않았습니다.

실기기 테스트 / 조회 오류·실기기 결과 / 온보딩 실패와 재검증 / 서버·DB 실행 기록

08 다음에 확인할 것

실기기 서버 연결부터 다시 확인해야 합니다. 시뮬레이터의 가입·저장은 통과했지만, 실제 iPhone에서는 같은 흐름을 끝내지 못했기 때문입니다. 연결 문제가 해결되면 저장 전후 화면과 HTTP 응답, DB 값을 함께 비교할 계획입니다.

순서	할 일	확인할 결과
1	실기기 서버 연결·가입·저장·동기화	차단 원인 확인, 화면·HTTP·DB의 값 일치
2	충돌 후 재저장 중 통신 실패·계정 전환	미저장 메모 유지, 중복 저장·다른 계정 노출 없음
3	단수 정보가 없는 항목의 감지·복구 절차	복원 근거 확인, 정상 기록과 소유 범위 유지
4	데이터가 늘어날 때 목록·도안 성능	데이터 규모와 응답 시간 기준을 정한 뒤 실측

이번에는 목록이 보이는지만 보지 않고 DB의 단수를 바꿔 실제 갱신을 확인했습니다. 저장 충돌도 409 응답에서 끝내지 않고, 기기에 남은 메모와 직접 재저장한 DB 값까지 확인했습니다.

테스트 코드와 실행 결과

사례별 코드·실행 결과